

6. [Process Synchronization]

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both. Write a program to solve the problem using process synchronization technique

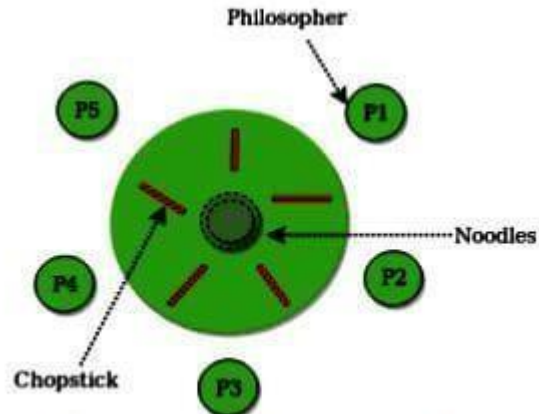


Fig: 1. Dining Philosopher Problem

CODE:

```
GNU nano 8.7 chopstick.c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];
pthread_t philosopher[N];

void* eat(void* arg) {
    int id = *(int*)arg;

    printf("Philosopher %d is thinking\n", id);
    sleep(1);

    sem_wait(&chopstick[id]);
    sem_wait(&chopstick[(id + 1) % N]);

    printf("Philosopher %d is eating\n", id);
    sleep(1);

    sem_post(&chopstick[id]);
    sem_post(&chopstick[(id + 1) % N]);

    printf("Philosopher %d finished eating\n", id);
    return NULL;
}

int main() {
    int i, id[N];

    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&philosopher[i], NULL, eat, &id[i]);
    }

    for (i = 0; i < N; i++)
        pthread_join(philosopher[i], NULL);

    return 0;
}
```

```
GNU nano 8.7 chopstick.c
#include <unistd.h>

#define N 5

sem_t chopstick[N];
pthread_t philosopher[N];

void* eat(void* arg) {
    int id = *(int*)arg;

    printf("Philosopher %d is thinking\n", id);
    sleep(1);

    sem_wait(&chopstick[id]);
    sem_wait(&chopstick[(id + 1) % N]);

    printf("Philosopher %d is eating\n", id);
    sleep(1);

    sem_post(&chopstick[id]);
    sem_post(&chopstick[(id + 1) % N]);

    printf("Philosopher %d finished eating\n", id);
    return NULL;
}

int main() {
    int i, id[N];

    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&philosopher[i], NULL, eat, &id[i]);
    }

    for (i = 0; i < N; i++)
        pthread_join(philosopher[i], NULL);

    return 0;
}
```

OUTPUT :

```
AnshuAnshu-Rog MSYS ~
$ nano dining.c
AnshuAnshu-Rog MSYS ~
$ gcc dining.c -o dining -pthread
AnshuAnshu-Rog MSYS ~
$ ./dining.c
./dining.c: line 13: int: command not found
./dining.c: line 14: sem_t: command not found
./dining.c: line 15: sem_t: command not found
./dining.c: line 17: syntax error near unexpected token '('
./dining.c: line 17: `int LEFT(int i) {'
AnshuAnshu-Rog MSYS ~
$ gcc dining.c -o dining -pthread
AnshuAnshu-Rog MSYS ~
$ ./dining
Philosopher 0 is Thinking
Philosopher 1 is Thinking
Philosopher 2 is Thinking
Philosopher 3 is Thinking
Philosopher 4 is Thinking
Philosopher 0 is Hungry
Philosopher 0 takes chopsticks 4 and 0
Philosopher 0 is Eating
Philosopher 1 is Hungry
Philosopher 2 is Hungry
Philosopher 2 takes chopsticks 1 and 2
Philosopher 2 is Eating
Philosopher 3 is Hungry
Philosopher 4 is Hungry
Philosopher 0 putting chopsticks 4 and 0 down
Philosopher 0 is Thinking
Philosopher 4 takes chopsticks 3 and 4
Philosopher 4 is Eating
Philosopher 2 putting chopsticks 1 and 2 down
Philosopher 2 is Thinking
Philosopher 1 takes chopsticks 0 and 1
Philosopher 1 is Eating
Philosopher 0 is Hungry
Philosopher 2 is Hungry
Philosopher 4 putting chopsticks 3 and 4 down
Philosopher 4 is Thinking
Philosopher 3 takes chopsticks 2 and 3
Philosopher 3 is Eating
Philosopher 1 putting chopsticks 0 and 1 down
Philosopher 1 is Thinking
Philosopher 0 takes chopsticks 4 and 0
Philosopher 0 is Eating
Philosopher 4 is Hungry
Philosopher 1 is Hungry
Philosopher 3 putting chopsticks 2 and 3 down
Philosopher 3 is Thinking
Philosopher 2 takes chopsticks 1 and 2
```